

Parallel Inference

July 27: AI Inference and Evaluation at Scale

Batching • sharding • throughput • latency • metric aggregation

INF

Big idea

Inference is a pipeline: data movement, model execution, and aggregation.



Student output

Build and benchmark a sharded inference engine.

Dong Li
UC Merced

Today's Roadmap

From preprocessing to parallel prediction and evaluation

01

Recap

process pools
and map–
reduce

02

Inference

latency,
throughput,
batching

03

Design

shards,
workers,
model loading

04

Evaluation

metric
aggregation
and
correctness

05

Lab

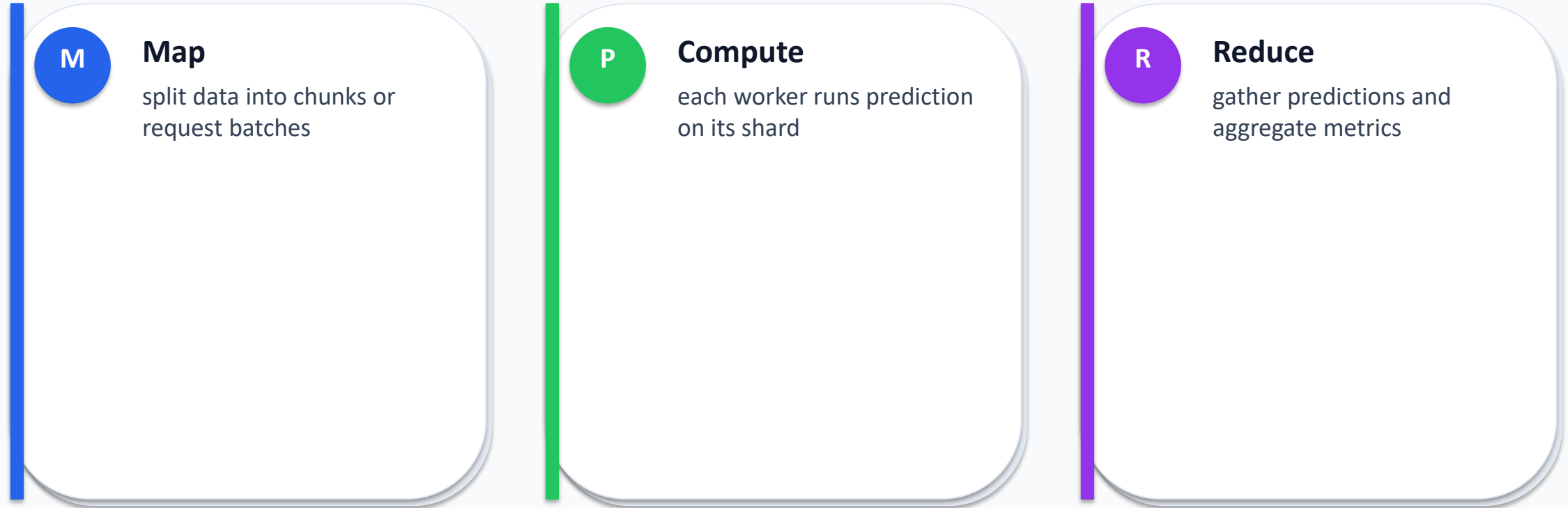
parallel
inference
benchmark

1. Recap and Motivation

Why inference deserves its own design

Day 3 Recap: Map–Reduce Thinking

Day 4 applies the same pattern to prediction and evaluation



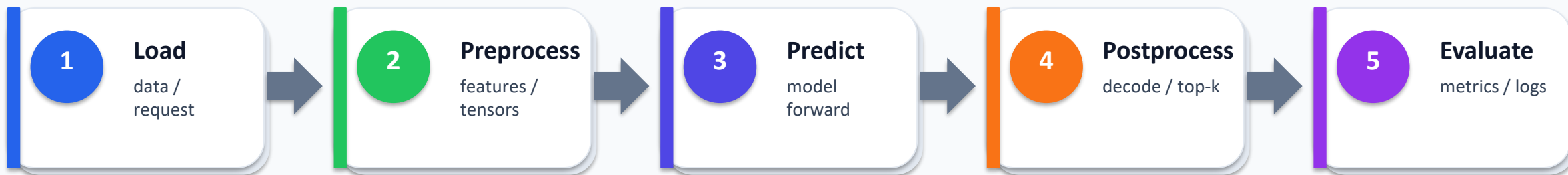
Inference is often embarrassingly parallel — but model loading and data movement matter

2. Inference Performance Concepts

Latency, throughput, batch size, and bottlenecks

Inference Pipeline

Every stage can become the bottleneck



latency per item

throughput items/sec

end-to-end metric

Throughput vs Latency

They are related but not the same metric

T

Throughput

How many samples or requests are completed per second. Good for batch jobs and offline evaluation.

L

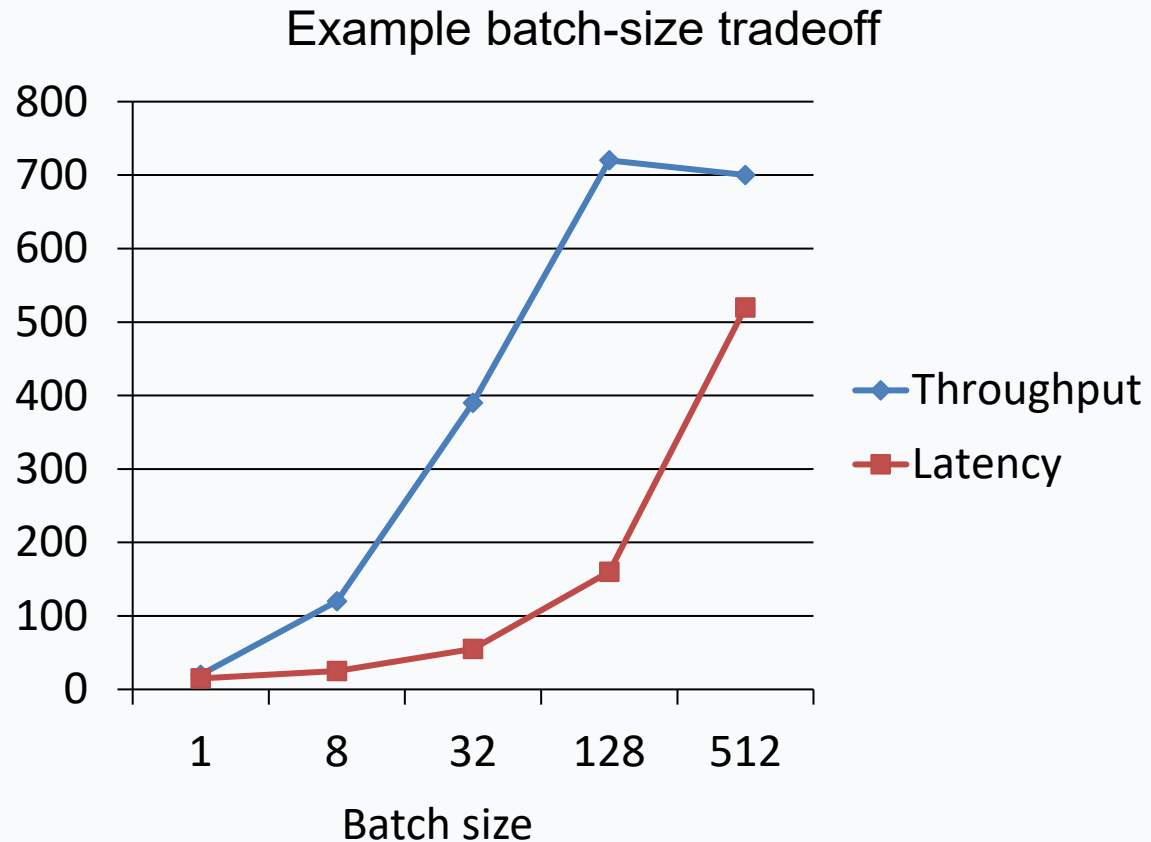
Latency

How long one sample or request waits from start to finish. Good for demos and real-time systems.

Batching usually improves throughput, but can increase waiting time for individual requests

Batch Size Tradeoff

Large batches improve efficiency — until memory or waiting hurts



1

Small batch

low waiting, low hardware utilization

✓

Medium batch

often best throughput/latency balance

!

Huge batch

memory pressure or unacceptable latency

PyTorch Inference Best Practices

Disable training behavior and gradient tracking

eval

model.eval ()

Sets modules like dropout and batch normalization into evaluation behavior.

grad

torch.no_grad ()

Disables gradient calculation and reduces memory for inference computations.

fast

torch.inference_mode ()

A stricter inference context that removes additional autograd overhead when safe.

scikit-learn Inference: Hidden Parallelism

Be careful not to oversubscribe CPU cores

jobs

n_jobs

Some estimators/utilities use joblib and expose n_jobs for high-level parallelism.

BLAS

BLAS/OpenMP

NumPy/SciPy/scikit-learn internals may already use lower-level thread pools.

!

Oversubscription

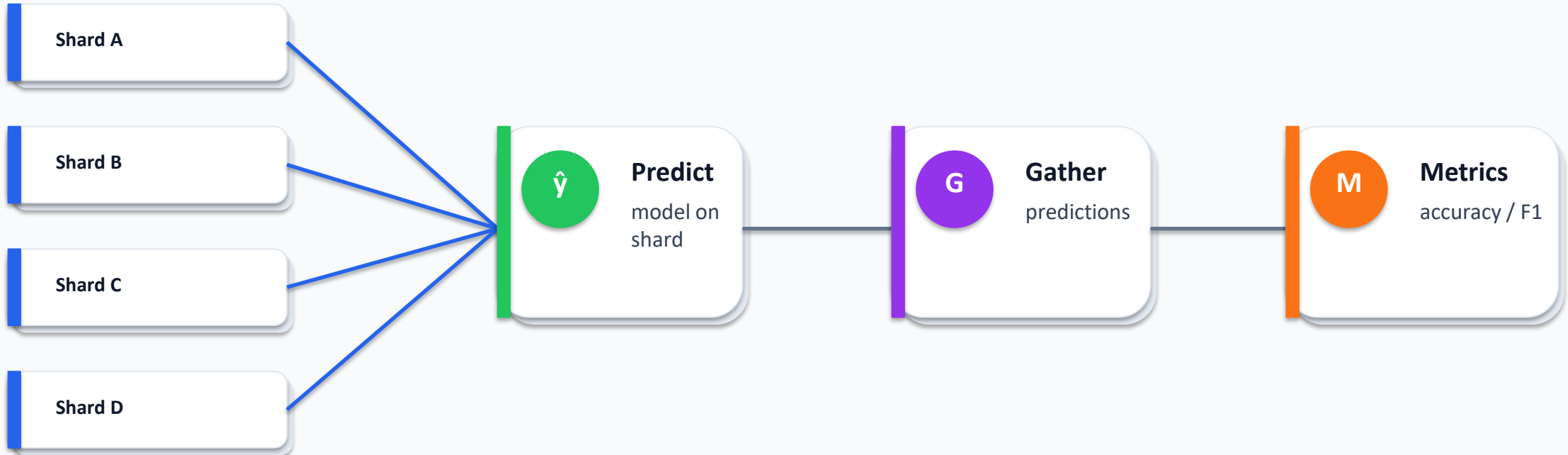
Outer Python parallelism plus inner native threads can create too many workers.

3. Parallel Inference Design Patterns

Shards, workers, model loading, and aggregation

Pattern 1: Sharded Batch Inference

Split dataset → predict chunks → gather predictions



Model Loading Strategy

Loading the model repeatedly can erase parallel speedup

X

Bad pattern

Each task loads the model again. Huge overhead if model load is slow or memory-heavy.

✓

Better pattern

Each worker loads the model once, then processes many chunks or requests.

Which Execution Model Should I Try?

The best choice depends on model and bottleneck

V

Vectorized batch predict

Often best for scikit-learn/NumPy models because internal native code is optimized.

T

Threads

Useful when prediction waits on I/O or model code releases the GIL.

P

Processes

Useful for CPU-heavy Python inference or independent model replicas.

When Parallel Inference Can Be Slower

More workers can create more overhead

1

Tiny batches

overhead dominates useful compute

2

Repeated model loading

startup cost dominates

3

Large models

memory pressure from replicas

4

Nested parallelism

too many threads

5

Bad sharding

load imbalance

6

Slow aggregation

metrics/postprocess bottleneck

4. Evaluation and Metrics Aggregation

Correctness before speed, metrics after gathering

Metric Aggregation Pattern

Workers produce predictions; main process computes global metrics

$\hat{y}$

Worker outputs

predicted labels,
probabilities, and chunk IDs

G

Gather

preserve order or keep IDs;
handle errors; combine
arrays

M

Metrics

accuracy, F1, confusion
matrix, latency, throughput

Correctness Checks

Parallel predictions must match the baseline

order

Order-preserving design

Use executor.map or return chunk IDs and sort results before metric computation.

check

Baseline comparison

Serial predictions and parallel predictions should match exactly or within tolerance.

Speedup does not matter if labels and metrics are wrong

Measurement Plan

Separate model quality from system performance

1

Quality

accuracy, F1, confusion matrix

2

Speed

runtime and samples/sec

3

Latency

mean, p50, p95 per batch/request

4

Resource

CPU cores, memory, batch size

5

Correctness

same predictions as baseline

6

Reproducibility

fixed seeds and run settings

5. Day 4 Lab

Build a parallel inference engine

Lab: Parallel Inference Engine

Goal: shard test data, predict in parallel, gather metrics



Deliverable: runtime table + speedup chart + accuracy correctness check

Lab Starter: Train a Small Classifier

Use scikit-learn digits so everyone can run the lab

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import numpy as np

X, y = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.35, random_state=42, stratify=y)

model = RandomForestClassifier(
    n_estimators=100, random_state=42, n_jobs=1)
model.fit(X_train, y_train)
```

Step 1: Serial Inference Baseline

Measure before parallelizing

```
import time
from sklearn.metrics import accuracy_score

def time_call(label, func):
    start = time.perf_counter()
    result = func()
    elapsed = time.perf_counter() - start
    print(f"{label}: {elapsed:.4f} seconds")
    return result, elapsed

serial_pred, serial_time = time_call(
    "serial predict", lambda: model.predict(X_test))
serial_acc = accuracy_score(y_test, serial_pred)
print("serial accuracy:", serial_acc)
```

Step 2: Shard the Test Set

Keep chunk IDs so results can be restored in order



```
def make_chunks(X, chunk_size):
    chunks = []
    for start in range(0, len(X), chunk_size):
        end = min(start + chunk_size, len(X))
        chunks.append({"chunk_id": len(chunks),
                      "start": start, "end": end,
                      "X": X[start:end]})

    return chunks

chunks = make_chunks(X_test, chunk_size=64)
print("number of chunks:", len(chunks))
```


Step 3: Worker Prediction Function

Each worker predicts one chunk and returns metadata



```
def predict_chunk(args):
    model, chunk = args
    preds = model.predict(chunk["X"])
    return {"chunk_id": chunk["chunk_id"],
            "start": chunk["start"],
            "end": chunk["end"],
            "preds": preds}

def combine_chunk_predictions(outputs, n_samples):
    y_pred = np.empty(n_samples, dtype=int)
    for out in outputs:
        y_pred[out["start"]:out["end"]] = out["preds"]
    return y_pred
```

Step 4A: Threaded Inference

Useful when model code releases GIL or waits on I/O



```
from concurrent.futures import ThreadPoolExecutor

def threaded_predict(model, X, workers=4, chunk_size=64):
    chunks = make_chunks(X, chunk_size)
    tasks = [(model, chunk) for chunk in chunks]
    with ThreadPoolExecutor(max_workers=workers) as ex:
        outputs = list(ex.map(predict_chunk, tasks))
    return combine_chunk_predictions(outputs, len(X))

thread_pred, thread_time = time_call(
    "threaded predict",
    lambda: threaded_predict(model, X_test, workers=4))
print("matches serial:", np.array_equal(thread_pred, serial_pred))
print("accuracy:", accuracy_score(y_test, thread_pred))
```

Step 4B: Process-Based Inference

Useful for compute-heavy independent replicas, but pickling can cost time

```
from concurrent.futures import ProcessPoolExecutor

def process_predict(model, X, workers=4, chunk_size=64):
    chunks = make_chunks(X, chunk_size)
    tasks = [(model, chunk) for chunk in chunks]
    with ProcessPoolExecutor(max_workers=workers) as ex:
        outputs = list(ex.map(predict_chunk, tasks, chunksize=1))
    return combine_chunk_predictions(outputs, len(X))

# Put process-pool calls inside main() in a script.
process_pred, process_time = time_call(
    "process predict",
    lambda: process_predict(model, X_test, workers=4))
print("matches serial:", np.array_equal(process_pred, serial_pred))
```

Step 5: Benchmark Workers and Batch Size

Find a measured sweet spot

```
def benchmark_inference(model, X_test, y_test, serial_pred, serial_time):
    rows = []
    for method in ["thread", "process"]:
        for workers in [1, 2, 4, 8]:
            for chunk_size in [16, 64, 256]:
                fn = threaded_predict if method == "thread" else process_predict
                start = time.perf_counter()
                pred = fn(model, X_test, workers=workers, chunk_size=chunk_size)
                elapsed = time.perf_counter() - start
                rows.append({"method": method, "workers": workers,
                           "chunk_size": chunk_size, "time": elapsed,
                           "speedup": serial_time / elapsed,
                           "samples_per_sec": len(X_test) / elapsed,
                           "accuracy": accuracy_score(y_test, pred),
                           "correct": np.array_equal(pred, serial_pred)})
            print(rows[-1])
    return rows
```

PyTorch Inference Pattern

For teams using neural networks



```
# Pattern for PyTorch inference
model.eval()
all_preds = []

with torch.inference_mode():      # or torch.no_grad()
    for batch in dataloader:
        inputs = batch["x"].to(device)
        logits = model(inputs)
        preds = logits.argmax(dim=1)
        all_preds.append(preds.cpu())

y_pred = torch.cat(all_preds).numpy()
```

Mini Challenge

Optimize and explain your inference engine

1

Run

Benchmark serial, threaded, and process inference.

2

Record

runtime, speedup, samples/sec, accuracy, correctness.

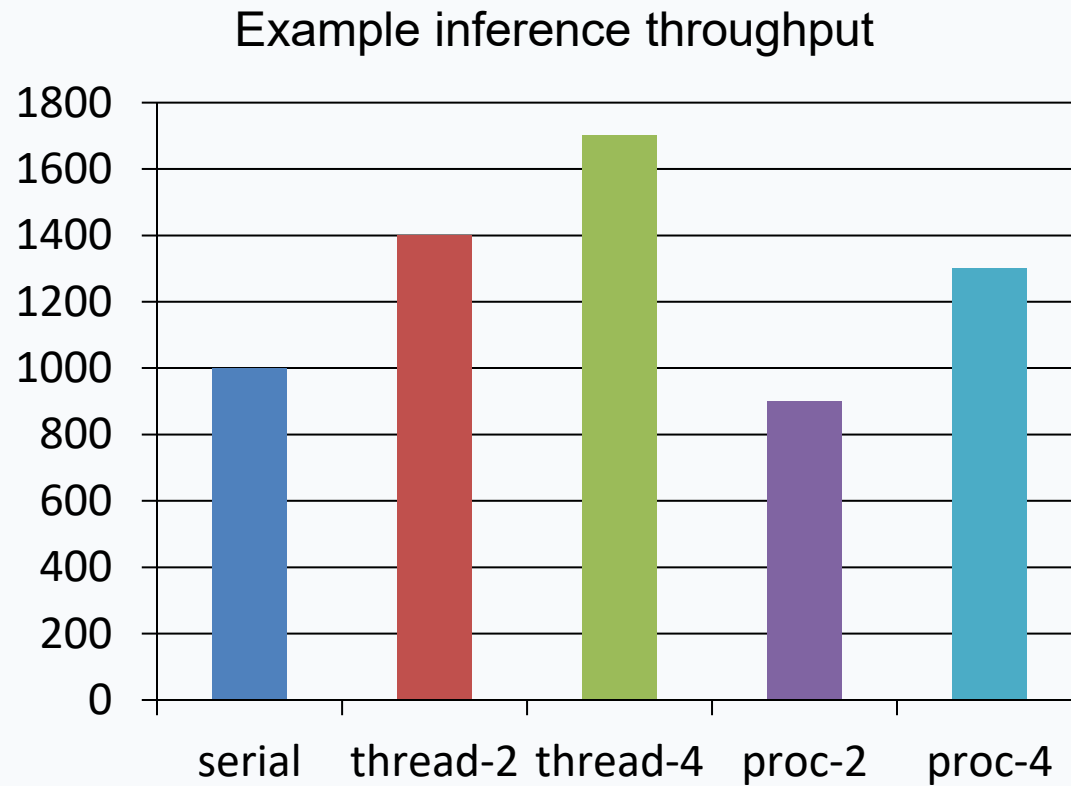
3

Explain

Why was one method faster? Did chunk size matter?

Expected Result Format

Students should report speed and model quality



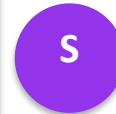
Correctness

Parallel predictions match serial baseline.



Quality

Accuracy/F1 stays unchanged.



Systems metric

Runtime, speedup, samples/sec, latency.

Exit Ticket

Short reflection + benchmark table

Q

Answer

- Throughput vs latency?
- Which method was fastest?
- Did predictions match serial?
- What bottleneck do you suspect?
- How could your project use sharded inference?

✓

Submit

- Code or notebook
- Runtime/speedup table
- Accuracy/correctness check
- One chart
- 1-paragraph explanation

References and Further Reading

Inference, futures, and parallel resource management

1. PyTorch: `no_grad` disables gradient calculation and reduces memory during inference.
2. PyTorch: `inference_mode` is analogous to `no_grad` and removes additional overhead when safe.
3. scikit-learn: estimators/utilities may use joblib, OpenMP, or BLAS parallelism; `n_jobs` controls joblib-managed parallelism.
4. Python `concurrent.futures`: Executor supports asynchronous calls via threads or processes; `map/submit/Future` are common patterns.